

Software Applications for Computer Science

Software Skills for the Real World

Today: Operation Systems, IT Security Essentials and more about CLIs

Structure of this seminar

1	Friday, September 11, 2026	Quarto/ RStudio/ APA
2	Friday, September 18, 2026	Quarto/ RStudio/ APA
3	Friday, September 25, 2026	Introduction and first hands-on example
4	Friday, October 2, 2026	Operating Systems, basic security and more about CLIs
5	Friday, October 9, 2026	AI ecosystem: LLMs, tokens, agents, harnesses and more
6	Friday, October 16, 2026	Using Git, Github and Claude Code for Creating a Website
7	Friday, October 23, 2026	Docker and containerization
	— — —MID TERMS— — —	After the Mid Terms it is your turn. Topics are not final yet!
8	Friday, November 6, 2026	OpenSource, Privacy and Security Aspects
9	Friday, November 13, 2026	Markdown Languages: HTML, CSS, Markdown
10	Friday, November 20, 2026	Claude Code, Codex, OpenCode
11	Friday, November 27, 2026	Cloud Computing Concepts like Infrastructure as a Service
12	Friday, December 4, 2026	Claude Design and Figma
13	Friday, December 11, 2026	Project and Knowledge Management: Jira and Confluence
14	Friday, December 18, 2026	IDEs and debuggers

Outline for Today

- Recap and Quiz
- Source vs Binary Machine Code
- OpenSource Software
 - The Wikipedia principle applied to Software
- Major OS Overview
 - Kernel Design & UNIX
- Live Terminal Session
- Different Kinds of Processors
- It Security Essentials
 - Phishing, authentication, 2FA and more

Recap

- Scope of this seminar is to give you an overview about all kinds of software applications or tools that IT people in the industry need
- Like a craftsman, you need to know your tools to do your work professionally
- Practical experience is key for getting an IT job
- Your homework
 - Setting up a GitHub account
 - Installing WSL if you have a Windows PC
- Terminal or CLI gives you more control on how to use your computer
- I probably won't do live coding sessions anymore

Recap Quiz

- What does CLI stand for?
- Which two of the three OS use the same commands in the terminal/CLI?
 - MacOS
 - Linux
 - Windows
- Does AI works better with CLI or graphical user interfaces and what is the reason for it?

Source Code and Binary Machine Code

```
#include <stdio.h>

int main()
{
    printf(
        "Codeforwin");
    return 0;
}
```



```
1010101110001100100
1111000111100011000
1010100011100010100
1011011101001111111
1000101010111001010
1110101101010101010
1010110001000111011
1010100011101010011
0101010100001101110
1011110101101011101
0001111111111100010
```

Source Code and Open Source

- **Source Code:** Written in a programming language (a human readable set of instructions)
 - Gets **translated** into **binary machine code** (zeros and ones) with a so called **compiler**
- For **Closed Source** software (like MacOS or Windows) the source code is not published
 - Only the **binary machine code is published**
 - Examples: MacOS and Windows
- **Open Source publishes both:** source and binary machine code
 - Everybody is allowed to co-create and modify the source code
 - Think of it as the Wikipedia for software
 - Example: Linux and Firefox

Open Source: Pros and Cons

Pros and Cons

- Pro
 - Free and transparent
 - 100%* trustworthy when it comes to **data privacy**
 - **No dependency on the USA and BigTech**
 - Easily modifiable
- Con
 - Sometimes worse usability for consumers (eg. OpenOffice vs Microsoft Office)
 - Sometimes financing or future is unclear and depends on BigTech (Google, Amazon etc.)





Permanent
Record

Edward
Snowden



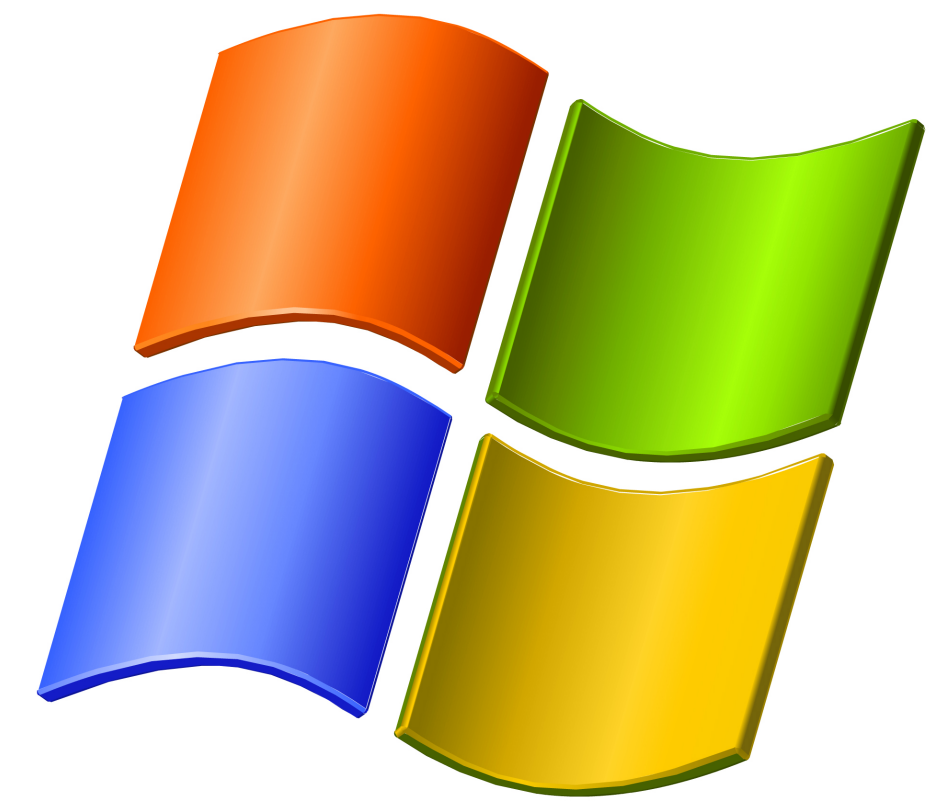
Internet's core infrastructure runs on open source software

- Linux runs the vast majority of web servers, cloud infrastructure, and supercomputers
- Two major web servers serve a large share of all websites
- Core internet protocols (HTTP and others) are open standards
- Databases & languages: MySQL, Python etc. — all open source and foundational to how websites run
- Big Tech companies (Google, Meta, Amazon) build much of their infrastructure on open source tools and also release major open source projects themselves

Major Operating Systems Overview

Windows

Operation Systems in Comparison



- **Market Presence:** The most widely used desktop operating system globally, running on hardware from virtually any manufacturer.
- **Strengths:** Unmatched software library.
- **Cost:** Paid license (usually bundled into the price of a pre-built PC).
- **Weaknesses:** Higher resource consumption; traditionally more vulnerable to malware and system clutter due to its massive user base.

MacOS

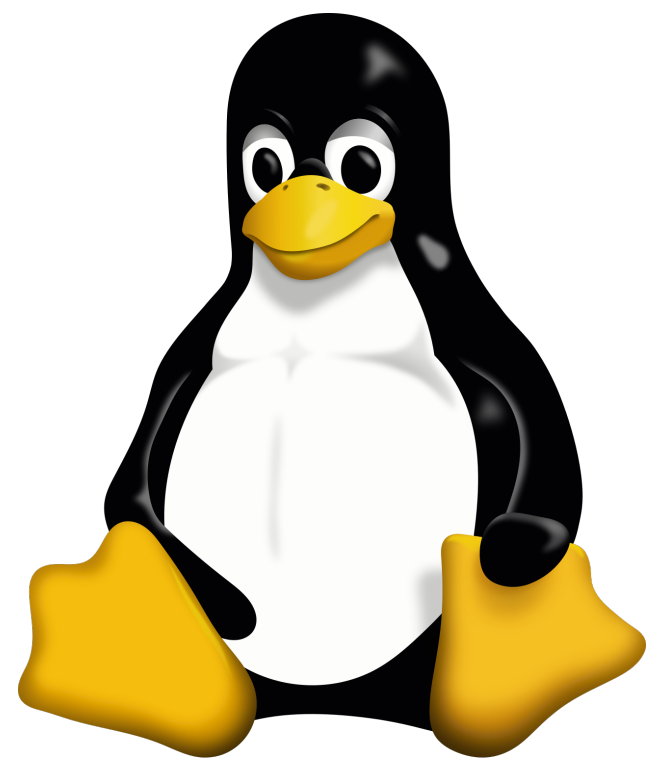
Operation Systems in Comparison



- **Market Presence:** Exclusive to Apple hardware (MacBook, iMac, Mac mini).
- **Strengths:** Highly intuitive user interface, exceptional stability, and seamless integration with the Apple ecosystem (iPhone, iPad). It is a top choice for creative professionals in video, audio, and photo editing. Hard- and Software from a single hand.
- **Cost:** The OS updates are free, but you must buy premium Apple hardware to use it.
- **Weaknesses:** Lacks hardware upgradability and offers limited flexibility if you want to build your own computer.

Linux

Operation Systems in Comparison



- **Market Presence:** An open-source operating system available in various versions called distributions (like Ubuntu, Fedora, or Linux Mint). It dominates servers but has a smaller desktop market share.
- **Strengths:** Completely free, lightweight (great for reviving old hardware), highly secure, and respects user privacy. It offers unlimited customization.
- **Cost:** 100% free.
- **Super stable:** Servers running Linux often have **uptimes of months or years** without rebooting
- **Weaknesses:** Steeper learning curve for beginners; some mainstream commercial software (like Adobe Creative Cloud) and certain video games do not run natively
- **Side Note:** Android is a modified version of Linux

The main developer of Linux: Linus Torvalds

- An ~~angry~~ emotional nerd that could possibly be very rich but decided to develop open source software instead



Linux Distributions

Linux comes in different flavors

- A Linux distro = Linux (the core, called the "kernel") + extra software bundled together to make a complete, usable OS
 - Most linux software can run on any distribution but the **packaging** is different
- Different distros exist because different people/companies package Linux differently for different needs
 - Ubuntu — beginner-friendly, widely used, good documentation
 - Debian — stable and reliable, often used for servers
 - Fedora — cutting-edge features, backed by Red Hat
 - Arch Linux — highly customizable, for advanced users
 - Linux Mint — simple, Windows-like experience for beginners

Package Managers are like an App Store for the terminal

- A package manager (apt, dnf, etc.) is like an app store: you search for software by name, and it downloads and installs it for you
- Packages, like apps, come with metadata — version info, description, dependencies — similar to how an app store shows requirements and details
- Both handle updates automatically — your package manager can update all installed software at once, just like an app store updates your apps
- Both pull from a central, curated source — a repository (repo), instead of the App Store or Google Play

Kernel Design

- What is a kernel? (quick reminder)
 - The core part of an OS that controls hardware, memory, and processes
- Monolithic Kernel
 - Everything is in one big piece
 - Fast, but one error can crash everything
 - Example: Linux

Kernel Design

- What is a kernel? (quick reminder)
 - The core part of an OS that controls hardware, memory, and processes
- Microkernel
 - Only the essentials are in the kernel
 - Rest runs separately and safely
 - Slower, but more stable

Kernel Design

- What is a kernel? (quick reminder)
 - The core part of an OS that controls hardware, memory, and processes
- Hybrid Kernel
 - Combines both ideas
 - Fast where it matters, safe where possible
 - Example: Windows, macOS
- In short
 - **Monolithic = fast**
 - **Microkernel = safe**
 - **Hybrid = a mix of both**

Unix and Unix-like

What MacOS, iOS, Android and Linux have in Common

- **Unix:** an operating system created in the 1970s, one of the first modern OS designs
- **Unix-like:** an operating system that follows the same ideas and structure as Unix, without being the original Unix code
- Every major OS but Windows is Unix-like today
 - Unix itself is very niche today

Benefits of Unix-like

- Everything is treated as a file (even hardware devices)
- Small, simple command-line tools that can be combined together
- Multi-user and multi-tasking from the start (many people/programs can use it at once)
- Strong focus on security and permissions (who can read/write/run what)
- Reputation for being stable

Difference between Unix-like and Windows

Design Philosophy

- Unix/Unix-like (e.g., Linux, macOS): **Originating from an academic/technical environment**, focused on simplicity, modularity, and the command line — "small programs that work together."
- Windows: Created for the mass market/end users, with a focus on graphical usability from the very beginning.

Difference between Unix-like and Windows

File System Structure

- Unix: single, unified file tree starting at root / — everything (even hardware devices) is treated as a file
- Windows: separate drive letters (C:, D:, etc.) for different storage devices

Difference between Unix-like and Windows

User Interface

- Unix: traditionally command-line first, GUI added on top (e.g., Linux desktops)
- Windows: GUI-first by design, though it has a command line too (PowerShell/CMD)

Difference between Unix-like and Windows

Typical Use Case

- Unix-like: dominant in servers, cloud, developers, embedded systems
- Windows: dominant in desktop/office use, gaming, business software

Major Operation Systems: Recap Quiz

- Always pick between Windows, MacOS and Linux
 - Which OS is open source?
 - Which OS is not Unix-like?
 - Which OS has a monolithic kernel design?

Live Session:
Experience the benefits of a
Unix-like terminal yourself

Where you find the instructions

- Go to the GitHub of this seminar
- Click at live_sessions
- Click at the file named week_4_first_steps_with_unix_terminal.md

Different Kind of Processors

32 vs 64 bit Processors

Different Version of Operating Systems

- Refers to how much data a processor can handle "at once"
- 64-bit processors can handle larger numbers and more memory (RAM) than 32-bit
- 32-bit systems can typically only use up to ~4 GB of RAM
- 64-bit systems can use far more RAM (practically, hundreds of GB+)
- Almost all modern computers and phones are 64-bit today
- Software also comes in 32-bit or 64-bit versions — a 64-bit OS can usually run older 32-bit programs, but not the other way around
- -> **If you can, always install the 64bit version**

x86 vs ARM Processors

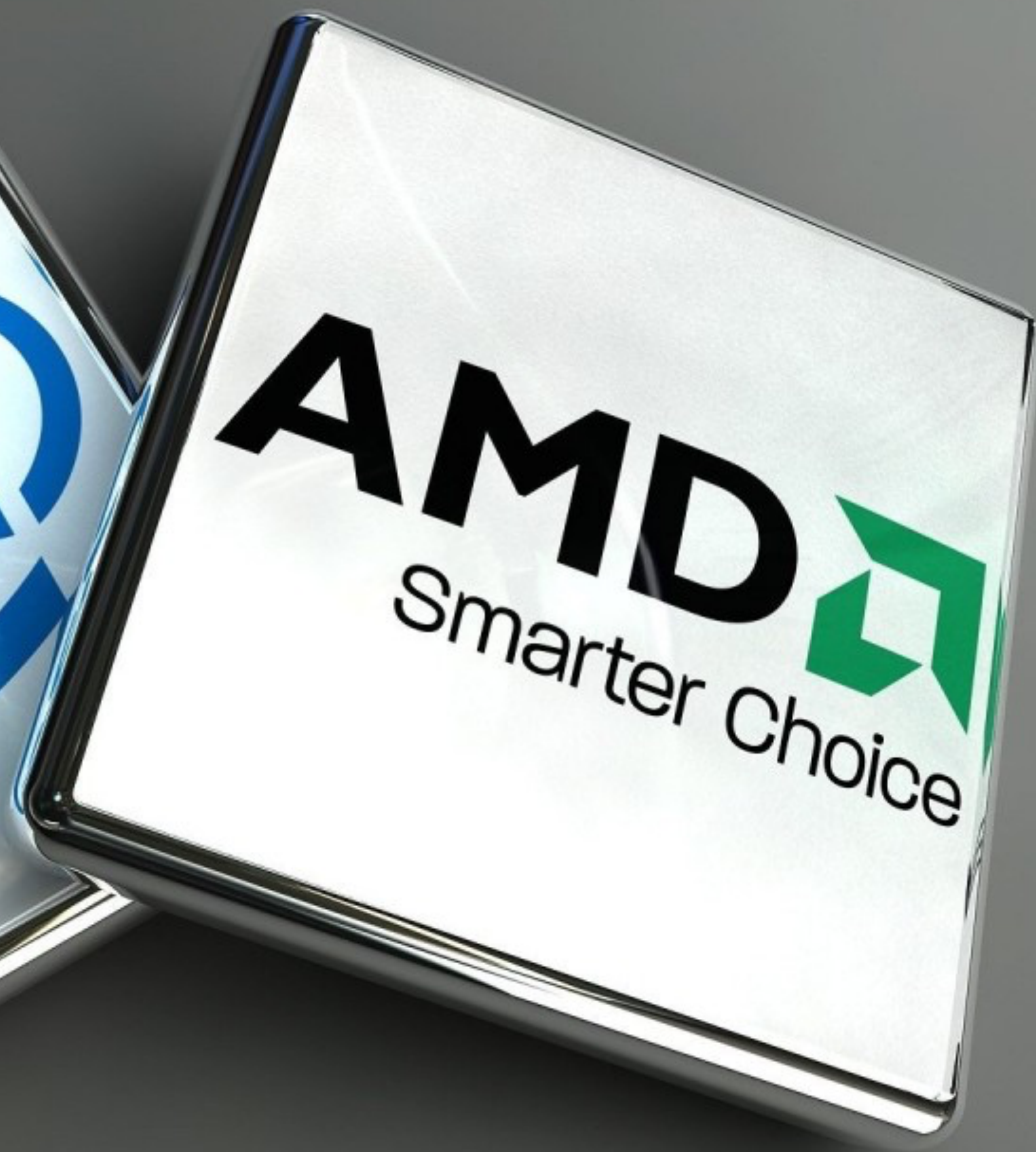
- Why "x86"?
 - Comes from Intel's early processor naming: 8086, 80286, 80386, 80486...
 - The common "86" ending led to the whole processor family being called x86
 - x86 became the standard architecture for PCs (Intel and AMD both use it)
 - x86-64 (or "x64") = the 64-bit extension of the original x86 architecture

x86 vs ARM Processors

- ARM
 - A different processor architecture (a different underlying design than x86)
 - Known for being very energy-efficient → ideal for battery-powered devices
 - Used in: smartphones, tablets, Raspberry Pi, and newer laptops (e.g., **Apple's M-series chips**)
 - Historically less powerful than x86 for heavy tasks, **but modern ARM chips have closed that gap significantly**
 - ARM licenses its designs to other companies (Apple, Qualcomm, Samsung), unlike Intel/AMD who make their own chips

All combined

	32 bit	64bit
x_86	Older computers	Modern computers
ARM	Older mobiles devices	Modern mobile devices (and since 2023 all Apple devices)



Why is 64-bit software sometimes labeled "AMD64"?

Often a Cause for Confusion

- AMD originally developed the 64-bit extension of the x86 architecture — hence the name AMD64
- This extension became the industry standard, and Intel adopted it too (Intel calls its version "Intel 64" or "EM64T," but it's technically compatible with AMD64)
- As a result, 64-bit software/operating systems (e.g., Linux distributions, Windows installers) are often labeled "amd64" — regardless of whether you have an AMD or Intel processor
- **Example: when downloading Ubuntu, you'll often see "amd64" in the filename for the 64-bit version — it runs fine on Intel CPUs too**

IT Security Essentials

IT Security Essentials

The biggest security threat sits between the chair and the keyboard

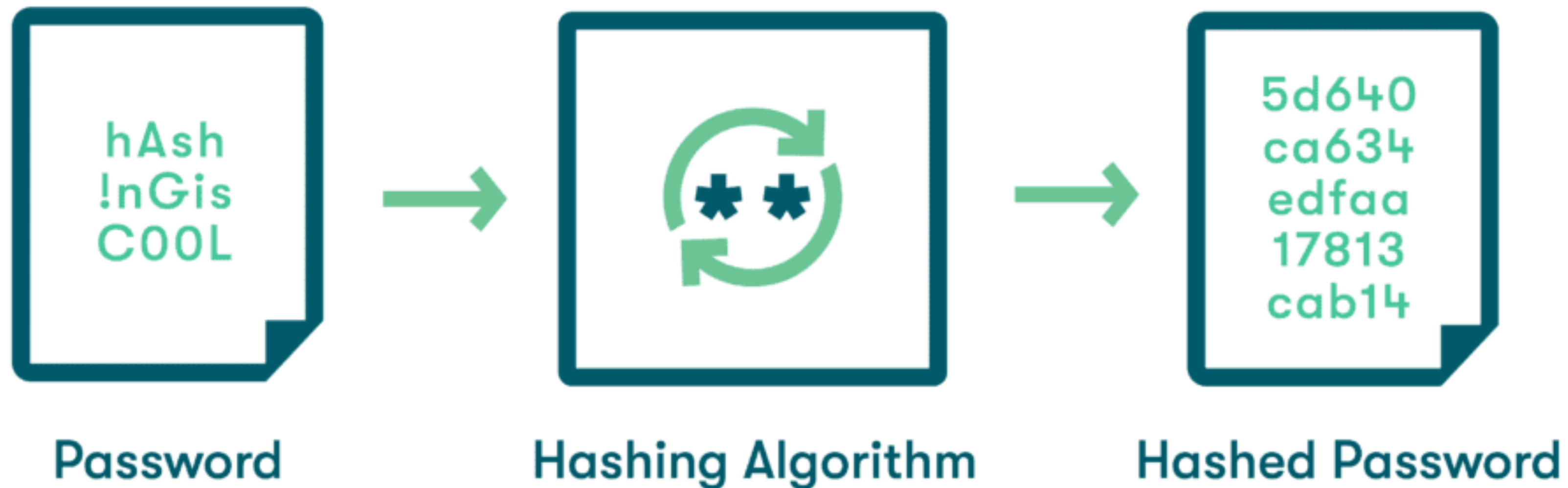
- What are best practices for passwords?
- What is phishing and how do you prevent it?
- What is the difference between passwords and passkeys?
- What is 2FA and how does it work?

Cryptography Essentials and Hashing

- The practice and study of techniques for secure communication in the presence of adversarial behavior
 - A secret language between you and your twin would be cryptography
- **Hashing** is the core building block in virtually every real-world encryption method

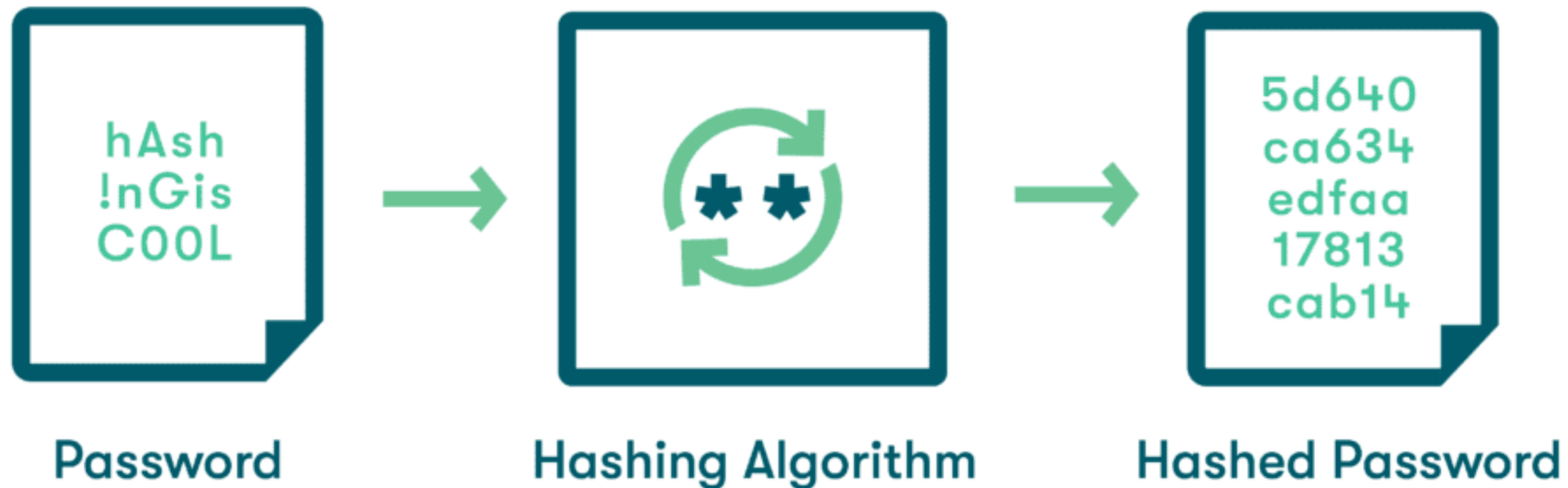
Trustworthy websites will never store your password!

Password Hashing



Only way to reconstruct you password from the hashed password is by brute force

Password Hashing



What is phishing and how to prevent it

Explained by an example

- Email pretends to be your bank's security team
- Red flags: threat, urgency, "Dear customer", unfamiliar sender domain
- The link leads to a look-alike site, the real domain is secure-verify.com
- You enter your login data and the attacker has it
- A password manager would not autofill, and 2FA/transaction approval blocks the attacker
- Safe habit: never use the link, open your bank's app or type the address yourself

Look-alike websites have a lock as well!

HTTPS alone is no sign for a trustworthy website



What are best practices for passwords and authentication?

Consider the pros and cons before you decide. It should be safe but also comfortable for everyday usage

- Use one password for all web sites?
- Use one password plus a custom part for all websites?
- Use a password manager with auto generated long passwords?
- Change your passwords every 6 months?

What are best practices for passwords and authentication?

Consider the pros and cons before you decide. It should be safe but also comfortable for everyday usage

- Use one password for all web sites?
- Use one password plus a custom part for all websites?
- **Use a password manager with auto generated long passwords?**
- Change your passwords every 6 months?

**Which is your most important
password?**

Passkeys

- Password: a secret you type in; passkey: a secure digital key stored on your device
- Nothing to remember, you unlock it with Face ID, fingerprint, or PIN
- The service never stores your secret, so there is nothing to steal from its servers
- A passkey only works on the real website, so phishing doesn't work
- No weak or reused passwords, because every account gets its own key automatically
- Biggest Downside: **Passkeys Depend on Your Device**
 - If you lose your device, you may lose access to your accounts
 - Some passkeys sync through the cloud, which is convenient but ties you to one provider

2 Factor Authentication

- 2FA means you prove who you are with two different things
- Factor 1: something you know (your password)
- Factor 2: something you have (your phone) or something you are (fingerprint)
- Even if someone steals your password, they still can't log in
- Authenticator apps show a 6-digit code that changes every 30 seconds

Why Authenticator Apps Work Offline

Do not loose your seed!

- Setup (once): you scan a QR code that contains a secret key
- Both your app and the website now know this secret
- The app mixes the secret key with the current time and calculates a 6-digit code
- The website does the same calculation on its side
- Same secret + same time = same code, so no internet connection needed
- The code changes every 30 seconds, so a stolen code quickly becomes useless

There is usually a trade-off between convenience and security: the more convenient a method is, the less protection it usually offers.

Security vs. convenience: Usability matters, too

- Security vs. convenience: there is always a trade-off
- But good design can reduce it: passkeys are both safer and easier than passwords
- The best security is the kind people actually use
- FaceID or Fingerprint is good for fast, everyday authentication
 - Ultrasonic 3D fingerprinting offers advantages over optical 2D methods
 - After a restart, your device always asks for your PIN or password first
 - Biometrics only work afterwards, as a shortcut

Outline for Today

- Recap and Quiz
- Source vs Binary Machine Code
- OpenSource Software
 - The Wikipedia principle applied to Software
- Major OS Overview
 - Kernel Design & UNIX
- Live Terminal Session
- Different Kinds of Processors
- It Security Essentials
 - Phishing, authentication, 2FA and more

See you next week